UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

IF 451
Advance Web Programming

Meet10 - REST API with Express and MySQL Database

Wirawan Istiono, S.Kom., M.Kom

Meet 10

Objective:

Sub-CLO0714 - Students are able to implement Node JS Framework to build Web application (C3, C6)

OUTLINE

- *GET, POST, PUT, DELETE routes.*
- Data validation.

Introduction

~ Preparing database and table ~

Create Database and Table

- First, please create database with database name is "**dbtoko**"
- You can create database with phpMyAdmin or MySQL CLI (please see meet4 if you forgot)
- Create table "**ms_barang**" with detail

```
CREATE TABLE ms_barang (
  id INT AUTO_INCREMENT PRIMARY KEY,
  kode_barang VARCHAR(20) NOT NULL,
  nama_barang VARCHAR(100) NOT NULL,
  harga INT NOT NULL,
  qty SMALLINT NOT NULL
);
```

Note: Don't forget to run apache and mysql if you use phpMyAdmin

Insert data

Insert some data into ms_barang, like the sample below

```
INSERT INTO ms_barang (kode_barang, nama_barang, harga, qty) VALUES
('111', 'Pensil', 5000, 12),
('112', 'Buku', 5000, 40),
('113', 'Penggaris', 5700, 34),
('114', 'Penghapus', 4500, 57),
('115', 'Pulpen', 8000, 130)
```

←T→			id	kode_barang	nama_barang	harga	qty
☐	Edit	Copy Delete	1	111	Pensil	5000	12
☐	Edit	Copy Delete	2	112	Buku	5000	40
☐	Edit	Copy Delete	3	113	Penggaris	5700	34
☐	Edit	Copy Delete	4	114	Penghapus	4500	57
☐	Edit	Copy Delete	5	115	Pulpen	8000	130

Install Dependencies

Don’t forget to install dependencies, with the command:

```
npm init -y
npm install express mysql2
```

GET, POST, PUT, DELETE routes.

What is REST API with Express and MySQL Database?

REST API (Representational State Transfer API) is a way to build web services that follow standard HTTP methods like:

- GET → read data
- POST → create new data
- PUT → update existing data
- DELETE → remove data

Create Connection Express → MySQL

Create new file, with name "**koneksi.js**", and write the code beside:

```
const mysql = require("mysql2");

const db = mysql.createConnection({
    host:"localhost",
    user:"root",
    password:"",
    database:"dbtoko"
});

db.connect(err => {
    if(err) {
        throw err;
    } else {
        console.log("koneksi berhasil");
    }
});

module.exports = db;
```

GET → read data (Single)

To get single data from database ms_barang, you can write code like the sample beside in the "app.js" files

```
const express = require("express");
const app = express();
const db = require("./koneksi");

app.get("/", (req, res) => {
    db.query("SELECT * FROM ms_barang", (err, results) => {
        if (err) {
            return res.status(500).send(err);
        }
        console.log("banyak data: " + results.length);
        let hasil = "";
        hasil = results[0].id + " | ";
        hasil += results[0].kode_barang + " | ";
        hasil += results[0].nama_barang + " | ";
        hasil += results[0].harga + " | ";
        hasil += results[0].qty + " | ";
        res.send(hasil);
    });
});

app.listen(3000, ()=>{ console.log("http://localhost:3000"); });
```

GET → read data (Multiple) + Border

To get multiple data from database ms_barang, you can write code like the sample beside in the "app.js" files

```
const express = require("express");
const app = express();
const db = require("./koneksi");

app.get("/", (req, res) => {
    db.query("SELECT * FROM ms_barang", (err, results) => {
        if (err) {
            return res.status(500).send(err);
        }
        let hasil = "";
        hasil += "<table border='1' rules='all'>";
        results.forEach((elm)=> {
            hasil += "<tr>";
            hasil += "<td>" + elm.id + "</td>";
            hasil += "<td>" + elm.kode_barang + "</td>";
            hasil += "<td>" + elm.nama_barang + "</td>";
            hasil += "<td>" + elm.harga + "</td>";
            hasil += "<td>" + elm.qty + "</td>";
            hasil += "</tr>";
        });
        hasil += "</table>";
        res.send(hasil);
    });
});

app.listen(3000, ()=>{ console.log("http://localhost:3000"); });
```

Send parameter POST, PUT, DELETE routes via Postman

What is Postman?

Postman is an API platform used by developers, QA engineers, and DevOps teams to build, test, and document APIs. It provides a user-friendly interface to send HTTP requests to APIs and analyze the server's responses, simplifying the entire API development lifecycle, from design and testing to collaboration and automation.

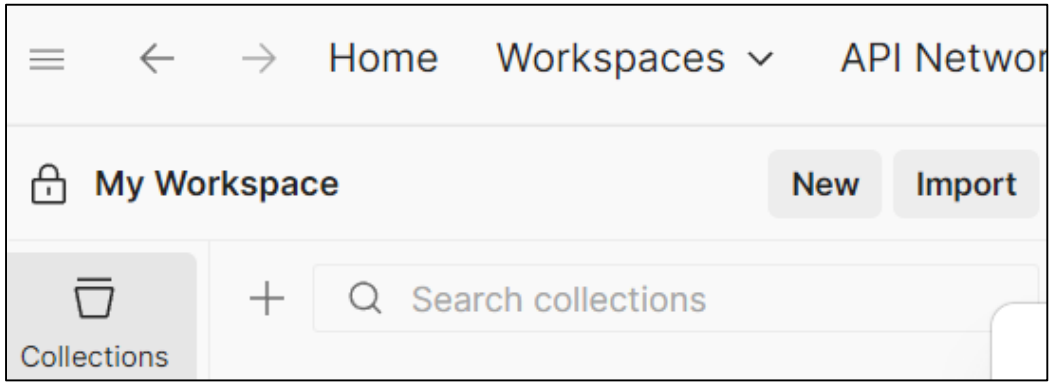
POSTMAN

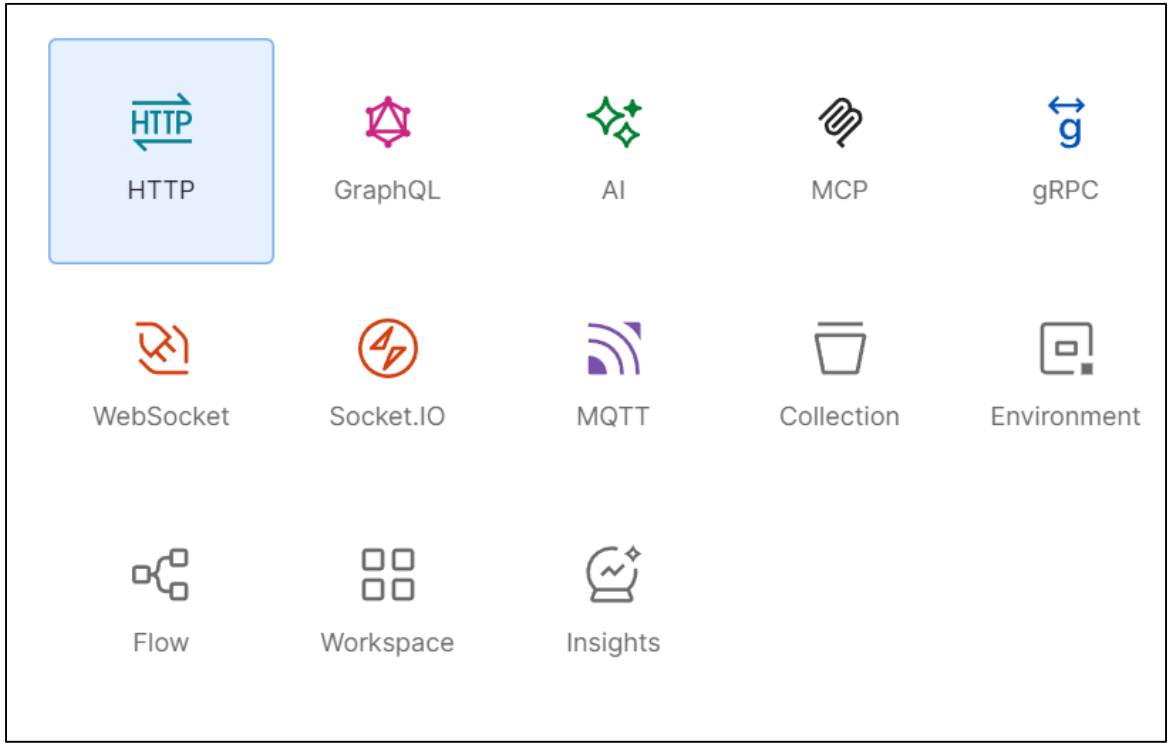
Install Postman

- To install postman, you can visit:

https://www.postman.com/downloads/

- After installation, in POSTMAN collections menu, choose New button - HTTP





POST routes → create new data (1/2)

- Add the sample code beside in your ***app.js*** to add new data to your **ms_barang** database.

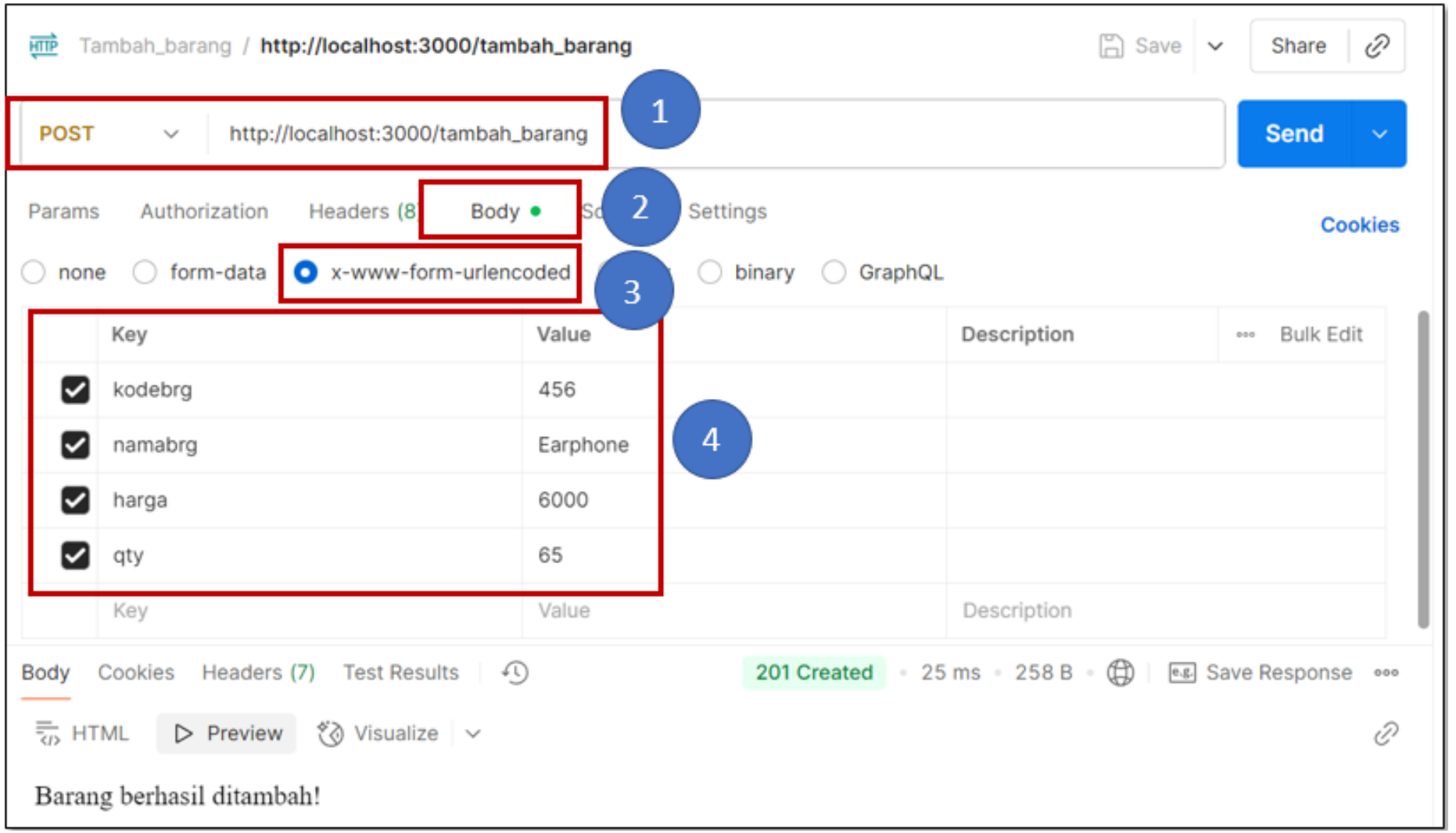
```
app.use(express.urlencoded({ extended: true }));

app.post("/tambah_barang", (req, res) => {
    const datapost = req.body;
    console.log("Barang yang akan ditambah: " + datapost.namabrg);

    db.query(
        "INSERT INTO ms_barang (kode_barang, nama_barang, harga, qty)
VALUES (?, ?, ?,?)",
        [datapost.kodebrg, datapost.namabrg, datapost.harga,
datapost.qty],
        (err, result) => {
        if (err) {
            return res.status(500).send(err);
        } else {
            res.status(201).send("Barang berhasil ditambah!");
        }
    });
});
```

POST routes → create new data (2/2)

- Send data POST via postman, with detail like the sample beside.
- And click **Send** button



PUT routes → update existing data (1/2)

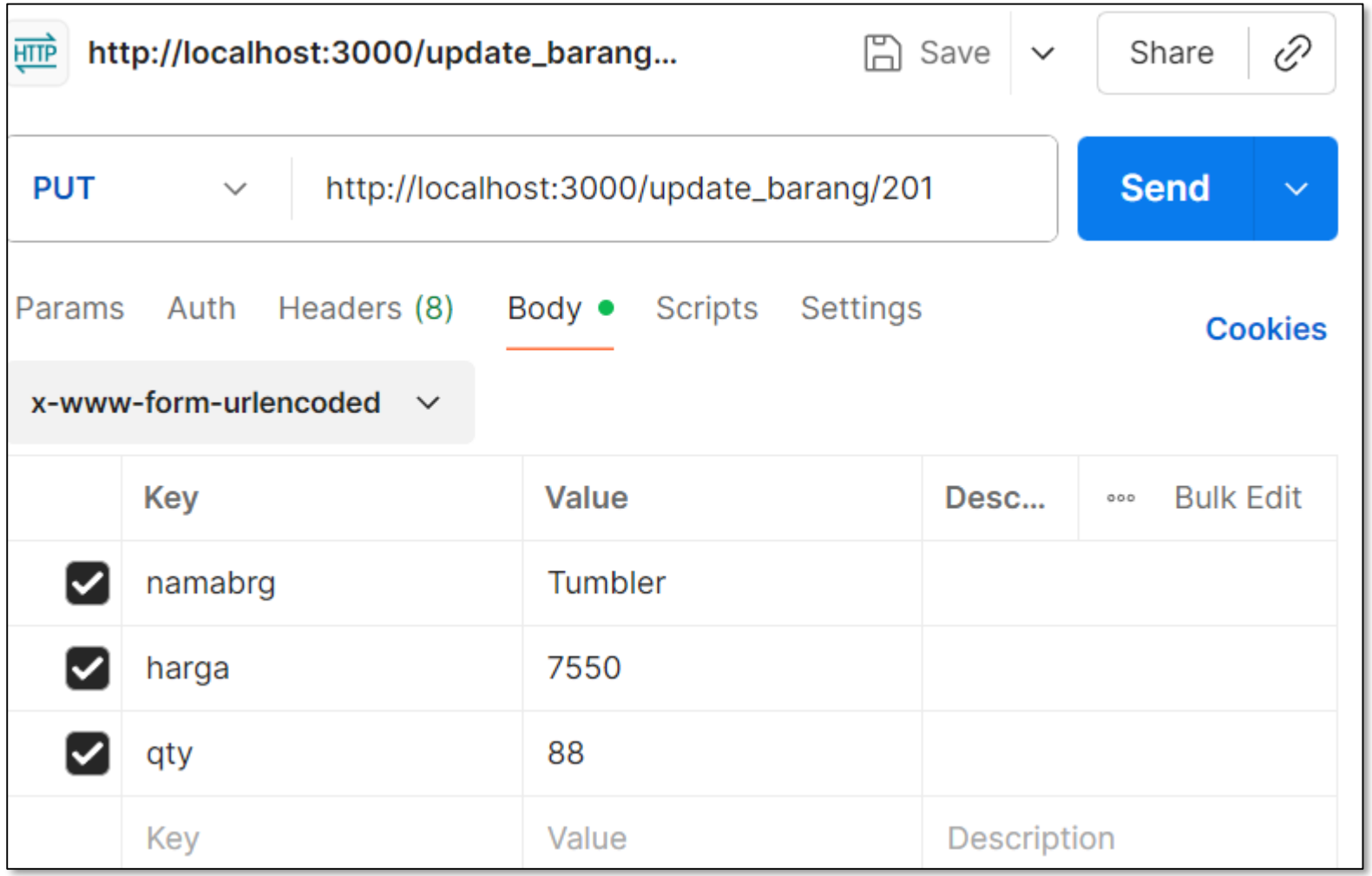
Please add the sample code beside into you app.js

```
app.put("/update_barang/:kode", (req, res) => {
    const kodebrg = req.params.kode;
    const dataPut = req.body;
    console.log("Barang yang akan diedit: " + dataPut.namabrg);

    db.query(
        "UPDATE students SET nama_barang=?, harga=?, qty=? WHERE kode_barang=?",
        [dataPut.namabrg, dataPut.harga, dataPut.qty, kodebrg],
        (err, result) => {
            if (err) {
                return res.status(500).send(err);
            } else if (result.affectedRows === 0) {
                return res.status(404).send(“Barang not found");
            } else {
                res.send("Barang Berhasil Update!");
            }
        }
    );
});
```

PUT routes → update existing data (2/2)

- Send data PUT via postman, with detail like the sample beside.
- And click **Send** button



POST vs PUT

POST → non-idempotent

- If you send a request multiple times, the results may vary.
- Example: POST to /add_barang → each time, a new entry will be added.

PUT → idempotent

- If you send the same request multiple times, the results will remain the same.
- Example: PUT to /update_barang/1 with the data {name: "Flashdisk"} → no matter how many times it is sent, the item id=1 will still be named "Flashdisk".

DELETE → remove data (1/2)

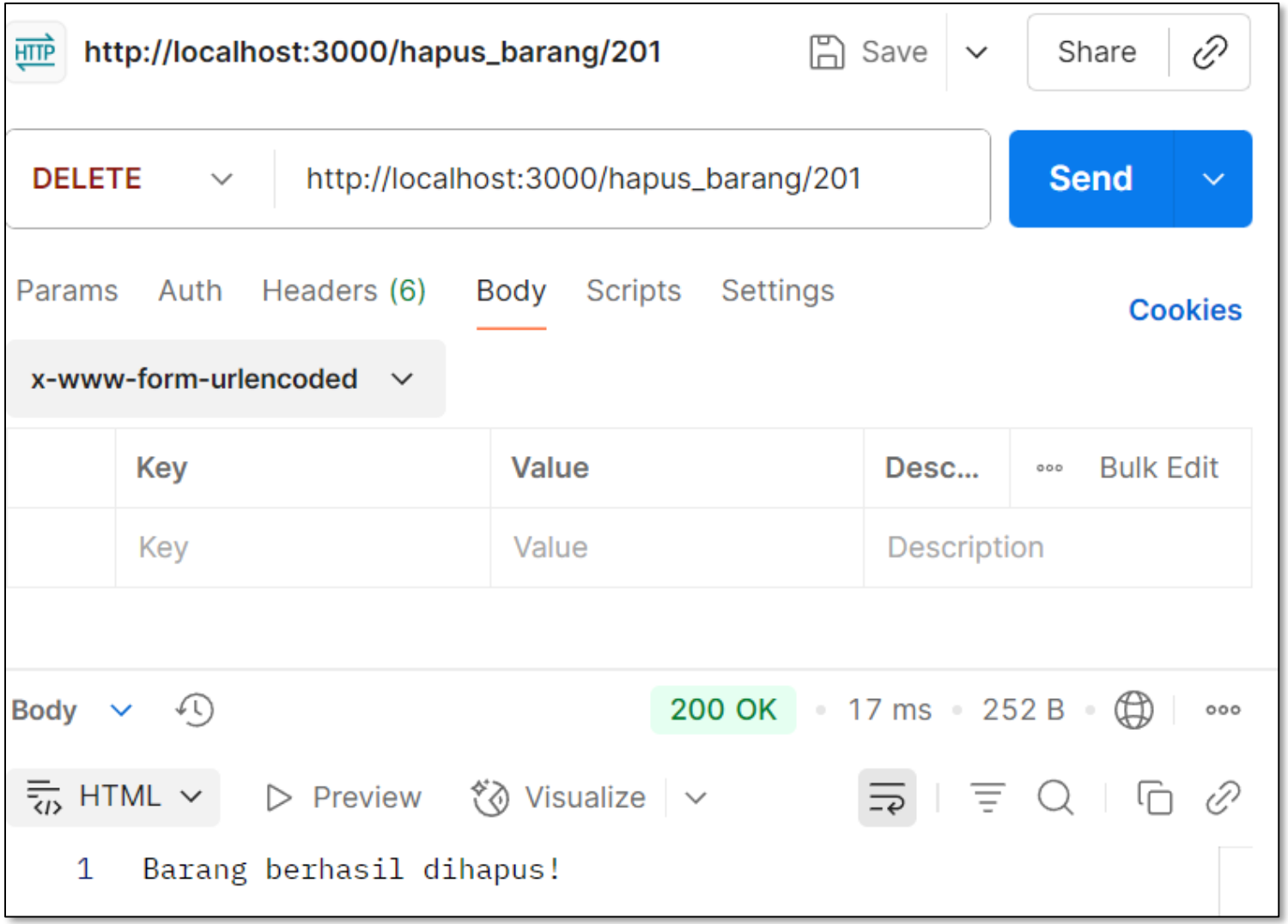
Please add the sample code beside into you app.js

```
app.delete("/hapus_barang/:kode", (req, res) => {
    const kodebrg = req.params.kode;
    console.log("Barang yang akan dihapus: " + kodebrg);

    db.query("DELETE FROM ms_barang WHERE kode_barang = ?",
    [kodebrg],
    (err, result) => {
        if (err) {
            return res.status(500).send(err);
        } else if (result.affectedRows === 0) {
            return res.status(404).send("Barang not found");
        } else {
            res.send("Barang berhasil dihapus!");
        }
    });
});
```

DELETE → remove data (2/2)

- Send data DELETE via postman, with detail like the sample beside.
- And click **Send** button



Data validation

Data Validation

To validate the data, you can add this code in you app.js before Insert, Update or Delete data

```
if (!kodebrg || !namabrg || !harga || !qty) {
  return res.status(400).send("All fields are required");
}
```

Or you can add your code to validate your data before CRUD operations.

NEXT WEEK'S OUTLINE

- Connecting Express API to ReactJS

REFERENCES

- B. Lawson and J. Encinas, Node.js Web Development, 6th ed., Packt Publishing, 2023
- R. Raj, Learning Node.js Development, Packt Publishing, 2018.
- E. Cantelon, M. Harter, T. Holowaychuk, and N. Rajlich, Node.js in Action, 2nd ed., Manning Publications, 2017.
- Maulana, a., bau, r.t.r., munawar, z., setiawan, r., aisa, s., istiono, w., irfan, a., pratama, y.a., ramadhany, e.d., pomalingo, s. And permana, a.a., 2023. *Pemrograman web 101: memahami dasar-dasar untuk mengembangkan situs web*. Get press indonesia.
- The Node.js Foundation, "Node.js Documentation," [Online]. Available: https://nodejs.org/en/docs/. [Accessed: Jun. 5, 2025].
- Express.js Team, "Express.js Documentation," [Online]. Available: https://expressjs.com/. [Accessed: Jun. 5, 2025].

Visi

Menjadi Program Studi Strata Satu Informatika **unggulan** yang menghasilkan lulusan **berwawasan internasional** yang **kompeten** di bidang Ilmu Komputer (*Computer Science*), **berjiwa wirausaha** dan **berbudi pekerti luhur**.

INFORMATIKA
UMN
UNIVERSITAS MULTIMEDIA NUSANTARA

Misi

1. Menyelenggarakan pembelajaran dengan teknologi dan kurikulum terbaik serta didukung tenaga pengajar profesional.
2. Melaksanakan kegiatan penelitian di bidang Informatika untuk memajukan ilmu dan teknologi Informatika.
3. Melaksanakan kegiatan pengabdian kepada masyarakat berbasis ilmu dan teknologi Informatika dalam rangka mengamalkan ilmu dan teknologi Informatika.